

Production Supporting Systems in Factories

ระบบสนับสนุนการผลิตในโรงงานอุตสาหกรรม

Advanced Integration

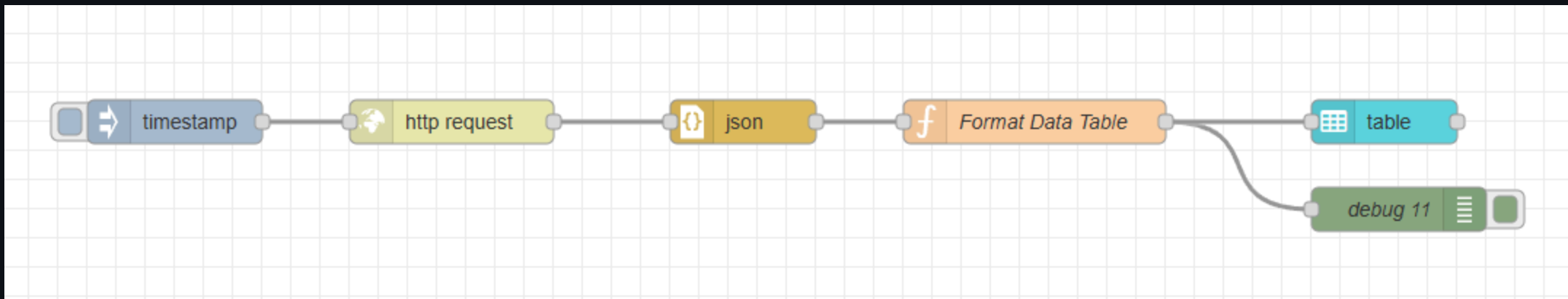
Retrieve Work Order from ERPNext and Create Stock Entry (Manufacture)
via HTTP Request

Information

```
KEY=  
SECRET=  
TOKEN=token ${KEY}:${SECRET}
```

```
BASE_URL=https://pmX-erpXX.iecmu.com  
URL1=${BASE_URL}/api/resource/Work Order?fields=["*"]&filters=[["docstatus","=","1"],["status","in",["Not Started", "In Process"]]]  
URL2=${BASE_URL}/api/method/make_stock_entry
```

Retrieve Work Order from ERPNext



HTTP GET Request

The screenshot displays the JMeter GUI with a test plan containing two test threads. The top thread consists of an 'http request' node (highlighted with a red box) followed by a 'json' node. The bottom thread consists of a 'Construct Payload' node followed by an 'ERP Creat' node. The 'Edit http request node' dialog is open on the right, showing the configuration for the selected 'http request' node. The 'Method' is set to 'GET', the 'URL' is 'https://pm4-erp30.iecmu.com/api/resource/Work Order', and the 'Payload' is set to 'Ignore'. The 'Return' dropdown is set to 'a UTF-8 string'. The 'Headers' section is expanded, showing an 'Authorization' header with the value 'token 477256912f164d4:84'.

Edit http request node

Delete Cancel Done

Properties

Method GET

URL https://pm4-erp30.iecmu.com/api/resource/Work Order

Payload Ignore

☐ Enable secure (SSL/TLS) connection

☐ Use authentication

☐ Enable connection keep-alive

☐ Use proxy

☐ Only send non-2xx responses to Catch node

☐ Disable strict HTTP parsing

Return a UTF-8 string

Headers

Authorization token 477256912f164d4:84

Format Data Table Function Code

```
const _data = msg.payload?.data;

// Validate that payload data is an array
if (!Array.isArray(_data)) {
  msg.payload = [];
  return msg;
}

// Format the data into a more readable table structure
const data = _data.map((d) => {
  return {
    Name: d["name"],
    Item: d["production_item"],
    "Target Qty": d["qty"],
    "To Be Produced": d["material_transferred_for_manufacturing"],
    Produced: d["produced_qty"],
  };
});

msg.payload = data;
return msg;
```

Table Node

The screenshot displays the Axure RP interface. On the left, a diagram on a grid shows a 'table' node (a light blue rounded rectangle with a table icon) connected to a 'debug 11' node (a green rounded rectangle with a list icon). A red rectangle highlights the 'table' node. Below it, a 'debug 12' node is also visible. On the right, the 'Properties' panel for the selected 'table' node is shown. The panel includes the following settings:

- Name:** Name
- Group:** [Home] Log
- Size:** auto
- Label:** optional label
- Class:** Optional CSS class name(s)
- Max Rows:** 0
- Action:** Replace
- Breakpoint:** defaults: Mobile (< 576px)
- Interaction:** None

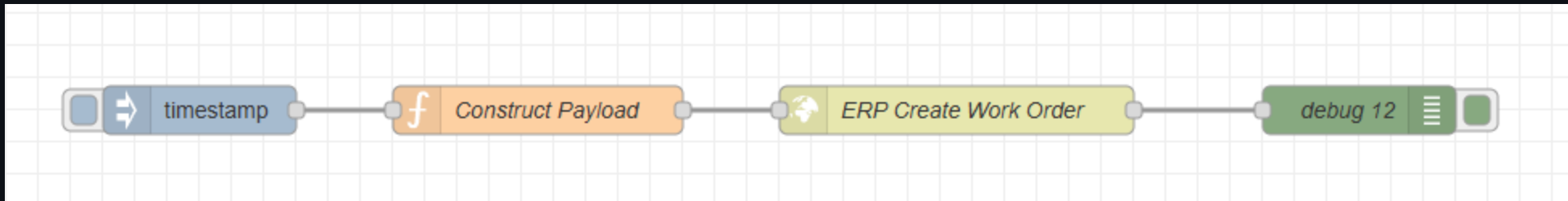
A red rectangle highlights the 'Action' and 'Breakpoint' settings in the Properties panel.

Create Stock Entry via HTTP **POST** Request

Create API Endpoint in ERPNext

- We need to create a custom API endpoint in ERPNext to handle the Stock Entry creation.
- Go to `Frappe Framework` -> `Build` -> `Server Script` -> `Add`
- Set `Script Type` to `API`
- Set `API Method` to `make_stock_entry`
 - If you change this name, remember to update the URL in Node-RED accordingly.
- Set `Script` to the following [code](#)

Node-RED Flow



Construct Payload Function Code

```
// Retrieve current work order and counter from flow context
// const current_work_order = flow.get("current_work_order");
// const counter = flow.get("counter")

// For demonstration purposes, we will hardcode these values
const current_work_order = "MFG-WO-2025-XXXX";
const counter = 1;

msg.payload = {
  work_order_id: current_work_order,
  purpose: "Manufacture",
  qty: counter,
};

return msg;
```

HTTP Request Node Configuration

